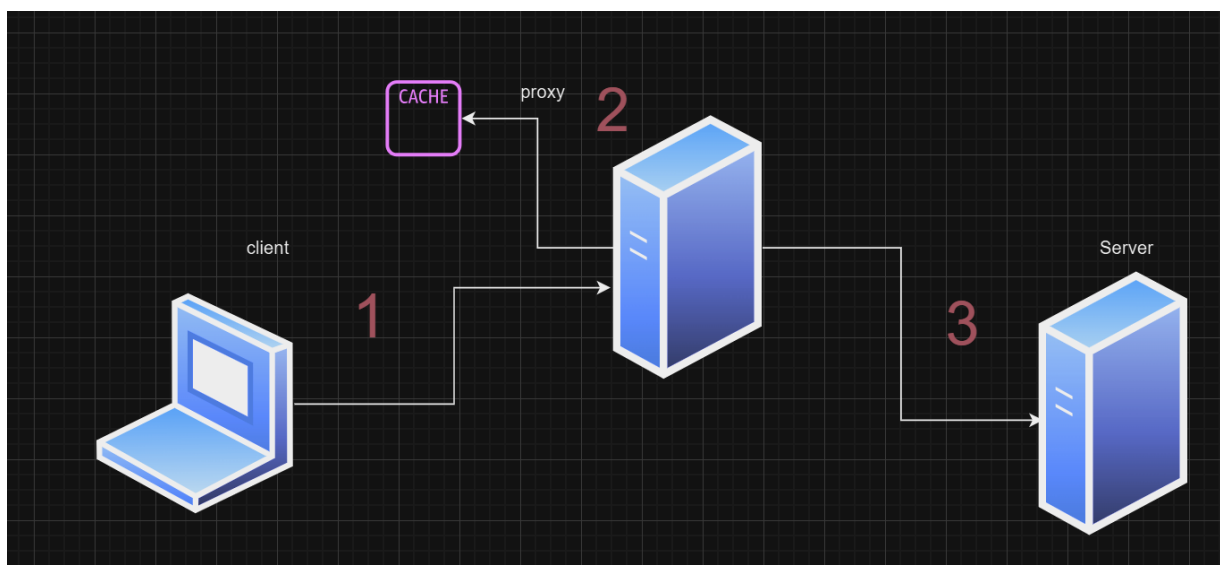


מטלה 2: רשתות ותקשורת, socket in python

1. צילומי מסך של החלקים שמילאנו עם הסברים. ראשית ננסה לציין את הקשרים בין כל הקבצים
 server.py מכיל את צד השרת
 client.py מכיל את צד הלקוח
 proxy.py פועל כמתווך בין client.py לserver.py

הוספנו דיאגרמה שהכנו בdrow.io על מנת שנוכל להבין את המתרחש, קליינט שולח בקשה כלשהיא (1) אם הבקשה שמורה ב cache אז נקבל תשובה חזרה ישירות מהפרוקסי (2) אם לא נשלח את הבקשה לשרת ולאחר מכן נשמור את התשובה לבקשה הזו בפרוקסי ונשלח בחזרה את התגובה לקליינט.



נתחיל בהשלמת הקובץ client.py:

אנחנו מבצעים את השינוי בתוך הפונקציה client שתפקידה לבצע את התקשורת בין הלקוח לשרת לקחנו את רשימת הביטויים המופיעה בהערות והכנסו אותן לתוך מערך. המשתמש בוחר פועלה בין 1-6 כל מספר מבטא פעולה כלשהיא ואם המשתמש בוחר 0 זה מעיד על סיום הקשר של הקליינט עם השרת או עם הפרוקסי. תחילה אנחנו מדפסים את ה"תפריט" שלנו ולאחר מכן קולטים מידע מהמשתמש, המבטא את הפעולה שאנחנו רוצים לעשות.

```

99     print("Choose an option to choose a calculation:")
100     print("1. max(2, 3) + 3")
101     print("2. 3 + ((4 * 2) / ((1 - 5) * (2 * 3)))")
102     print("3. ((1 + 2) ** (3 * 4)) / (5 * 6)")
103     print("4. -(-(1 + (2 + 3)) ** -(4 + 5)))")
104     print("5. max(2, (3 * 4), log(e), (6 * 7), (9 / 8))")
105     print("6. 4 * (3 + 5)")
106     print("0. Disconnect from the proxy/server")
107
108     user_choice = input("Enter choice or expression: ").strip()
109

```

```

109
110     while user_choice != "0":
111         try:
112             if user_choice.isdigit() and int(user_choice) in range(1, 7):
113                 # Use predefined examples based on user input
114                 predefined_expressions = [
115                     mul_b(
116                         div_b(sin_f(max_f(2, mul_b(3, 4), 5, mul_b(6, div_b(mul_b(7, 8), 9))), div_b(10, 11))), 12),
117                         13),
118                     add_b(max_f(2, 3), 3),
119                     add_b(3, div_b(mul_b(4, 2), pow_b(sub_b(1, 5), pow_b(2, 3)))),
120                     div_b(pow_b(add_b(1, 2), mul_b(3, 4)), mul_b(5, 6)),
121                     neg_u(neg_u(pow_b(add_b(1, add_b(2, 3)), neg_u(add_b(4, 5))))) ,
122                     max_f(2, mul_b(3, 4), log_f(e_c), mul_b(6, 7), div_b(9, 8))
123                 ]
124                 expr = predefined_expressions[int(user_choice) - 1]
125
126             show_steps = True # Request the steps of the calculation
127             cache_result = True # Request to cache the result of the calculation

```

כל עוד הקלט לא שווה ל0 נמשיך עם הבקשות שלנו מהקליינט לשרת או לפרוקסי , אם הקלט שלי הוא 0 זה מורה על כך שהקליינט רוצה להפסיק לשלוח בקשות .

הכנסו את כל הפעולות לסוג של מערך וכל פעם נחזיר את האינדקס -1 כי מתחילים לספור מ 0 וכך נקבל את הפעולה הרצויה על ידי המשתמש.

נמשיך בהשלמת הקובץ `server.py`:

לפי הסימונים ניתן לראות כי ההשלמה הראשונה נמצאת בפונקציה `server` שתפקידה להפעיל את שרת TCP שמאזין לבקשות בין לקוחות

מגדירה Socket TCP עם פרמטרים:

- AF_INET: שימוש בפרוטוקול IPv4

- SOCK_STREAM: תקשורת מבוססת TCP

לסוקט שיצרנו קוראים `server_socket`

השלמת הקוד הראשונה :

```

81 def server(host: str, port: int) -> None:
82     # socket(socket.AF_INET, socket.SOCK_STREAM)
83     # (1) AF_INET is the address family for IPv4 (Address Family)
84     # (2) SOCK_STREAM is the socket type for TCP (Socket Type) - [SOCK_DGRAM is the socket type for UDP]
85     # Note: context manager ('with' keyword) closes the socket when the block is exited
86     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
87         # SO_REUSEADDR is a socket option that allows the socket to be bound to an address that is already in use.
88         server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
89         # Prepare the server socket
90         # * Fill in start (1)
91         # * Fill in end (1)
92
93         #####
94         server_socket.bind((host, port)) # binding to our server socket an IP address and a port number.
95         server_socket.listen() # making our server listen for incoming TCP requests.
96         #####
97

```

אנחנו עדיין בפונקציית `server` שורה 94 מקצה את הפורט והכתובת לשרת כך שיוכל לקבל חיבורים נכנסים מלקוחות (clients) דרך פרוטוקול TCP זה בעצם תהליך הקישור בין ה `host` וה `port` באמצעות ה `.bind`. שורה 95 השרת מתחיל להאזין לחיבורים נכנסים מלקוחות , באמצעות ה `.listen`.

השלמת הקוד השנייה :

```

99     threads = []
100     print(f"Listening on {host}:{port}")
101
102     while True:
103         try:
104             # Establish connection with client.
105
106             client_socket, address = server_socket.accept()# * Fill in start (2) # * Fill in end (2)
107

```

שורה 106 : כמו שראינו בתרגול הפונקציה המובנת `accept()` היא כמו פתיחת דלת לחיבור חדש מלקוח הפונקציה הזו מחזירה לנו אובייקט סוקט חדש שמייצג את החיבור עם הלקוח (`client_socket`) ו-`address` מייצג את כתובת ה-`ip` והפורט של הלקוח שנוצר איתו החיבור .

השלמה שלישית :

ההשלמה הזו נמצאת בפונקציה הנקראת `client_handler` המטפלת בבקשות של לקוח מסוים כלומר היא קוראת את הבקשה שנשלחה על ידי הלקוח היא מנסה לפרש את הבקשה שולחת את התגובה חזרה ללקוח ומסיימת את החיבור כאשר הלקוח סוגר את התקשורת .

```

120     def client_handler(client_socket: socket.socket, client_address: tuple[str, int]) -> None: 1 usage
121         '''
122         Function which handles client requests
123         '''
124         client_addr = f"{client_address[0]}:{client_address[1]}"
125         client_prefix = f"{{{client_addr}}}"
126         with client_socket: # closes the socket when the block is exited
127             print(f"Connection established with {client_addr}")
128             while True:
129
130                 data = client_socket.recv(api.BUFFER_SIZE) # * Fill in start (3) # * Fill in end (3)
131                 if not data: # * Change in start (1)
132                     break
133                 # * Change in end (1)

```

שורה 130 היא השינוי השלישי שלנו בקובץ ומשמעותו :
 ניהול תקשורת בין השרת ללקוח. היא משמשת כדי לקרוא את הבקשות שנשלחות לשרת ומאפשרת לעבור לשלב שבו מעבירים את הנתונים ומחזירים תגובה מתאימה
`recv` --- < מגדירה שצריך לחכות עד שיגיעו נתונים דרך הסוקט , אם לא מגיעים נתונים התוכנית תמתין עד שהלקוח ישלח מידע כלשהוא .
`api.BUFFER_SIZE` --- < כמה נתונים אפשר לקרוא .

השלמה רביעית :

```

155         f"{client_prefix} Sending response of length {len(response)} bytes")
156         #send thr response back to the client
157         #we sent it to the client and close the connection
158         # * Fill in start (4)
159         client_socket.sendall(response)
160         # * Fill in end (4)
161

```

שורה 159 משמעותה היא שהשרת שולח את הנתונים שנמצאים ב-`response`-ללקוח דרך הסוקט שנוצר עבורו (`client_socket`).
 ללא פקודה זו לא היו שולחים את התגובה **מהשרת ללקוח**. כלומר, הלקוח לא היה מקבל כל מידע מהשרת.
`response` --- < משתנה שמכיל את הנתונים שיישלחו ללקוח.
`sendAll` --- < שיטה של אובייקט מסוג סוקט שמיועדת לשליחת נתונים

ולבסוף נמשיך עם השלמת הקובץ proxy.py:

ההשלמה הראשונה מתרחשת בפונקציה proxy המקימה פרוקסי שמאזין ללקוחות כלומר כל בקשה מהלקוח מועברת לשרת דרך פרוקסי זה, תוך שימוש בקאש (שמירה של נתונים בזיכרון על מנת לחסוך זמן תגובה) אפשר לשים לב שבקוד הנתון כן יש יצירת הסוקט אך הוא עדין לא נפתח ולא מאזין ולכן נדאג לזה

```

76
77 def proxy(proxy_address: tuple[str, int], server_address: tuple[str, int]) -> None:
78     # socket(socket.AF_INET, socket.SOCK_STREAM)
79     # (1) AF_INET is the address family for IPv4 (Address Family)
80     # (2) SOCK_STREAM is the socket type for TCP (Socket Type) - [SOCK_DGRAM is the socket type for UDP]
81     # Note: context manager ('with' keyword) closes the socket when the block is exited
82     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as proxy_socket:
83         # SO_REUSEADDR is a socket option that allows the socket to be bound to an address that is already in use.
84         proxy_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
85
86         # Prepare the proxy socket
87         # * Fill in start (1)
88         proxy_socket.bind(proxy_address) # binding to our proxy socket an IP address and a port number.
89         proxy_socket.listen() # making our proxy listen for incoming TCP requests.
90         # * Fill in end (1)

```

בשורה 82 אנחנו יוצרים סוקט עבור קבלה של בקשות מלקוחות לסוקט נקרא proxy_socket
 שורה 88 אנחנו מגדירים לכך שהסוקט מוגדר לפעול בכתובת ובפורט מוגדרים מראש על ידי הפונקציה המובנת bind, בלי, bind הסוקט לא יכול לדעת באיזו כתובת/פורט הוא צריך לפעול.
 בשורה 89 אנחנו פותחים את הסוקט שיצרנו להאזנה כך שאם לקוח ישלח בקשה אזי הסוקט יהיה פתוח.
 בלי, listen, הסוקט לא יוכל לקבל חיבורים נכנסים.
 בהשלמה השנייה של הקוד אנחנו עדין באותה פונקציה כמו שניתן לשים לב חסר בקוד שלנו את קבלת החיבור והטיפול בו
 אנחנו משתמשים בפונקציה המובנת accept והיא ממינה ממתניה לחיבור נכנס מלקוח. כאשר לקוח מנסה להתחבר לפרוקסי, הפונקציה יוצרת **סוקט חדש** המייצג את החיבור עם הלקוח שלו קראו client_socket היא גם מחזירה את הכתובת ip של הלקוח ששלח את הבקשה והתחבר לסוקט

```

92     threads = []
93     print(f"Listening on {proxy_address[0]}:{proxy_address[1]}")
94
95     while True:
96         try:
97             # Establish connection with client.
98             # Accepts a connection from a client. Returns a new socket for the client communication
99             # (client_socket) and the client's address (client_address).
100            client_socket, client_address = proxy_socket.accept()
101            # * Fill in start (2) # * Fill in end (2)
102

```

השלמת הקוד השלישית נמצאת בפונקציה הנקראת client_handler שמטרתה לטפל בבקשת הלקוח ולהעביר אותה לשרת כלומר היא בודקת אם הבקשה זמינה בקאש אם כן היא שולחת את התשובה ללקוח

```

117 def client_handler(client_socket: socket.socket, client_address: tuple[str, int], server_address: tuple[str, int]) -> None:
118     """
119     Function which handles client requests
120     """
121     client_prefix = f"{client_address[0]}:{client_address[1]}"
122     with client_socket: # closes the socket when the block is exited
123         print(f"{client_prefix} Connected established")
124         while True:
125             # Receive data from the client
126
127             # * Fill in start (3)
128             data = data = client_socket.recv(api.BUFFER_SIZE) ## # Receives data from the client via the client_socket, up to the specified buffer size.
129             # * Fill in end (3)
130

```

בשורה 128 ---< recv היא פונקציה שקוראת נתונים שנשלחו מהלקוח דרך הסוקט client_socket.

וגודל המקסימום של הנתונים הנקלטים מוגדר על ידי api.BUFFER_SIZE. זהו החלק שבו השרת מקבל נתונים ישירות מהלקוח. שורת קוד זו מאפשרת להבין מה הלקוח מבקש או שולח ומאפשרת לפרוקסי או

לשרת להגיב בהתאם.

```

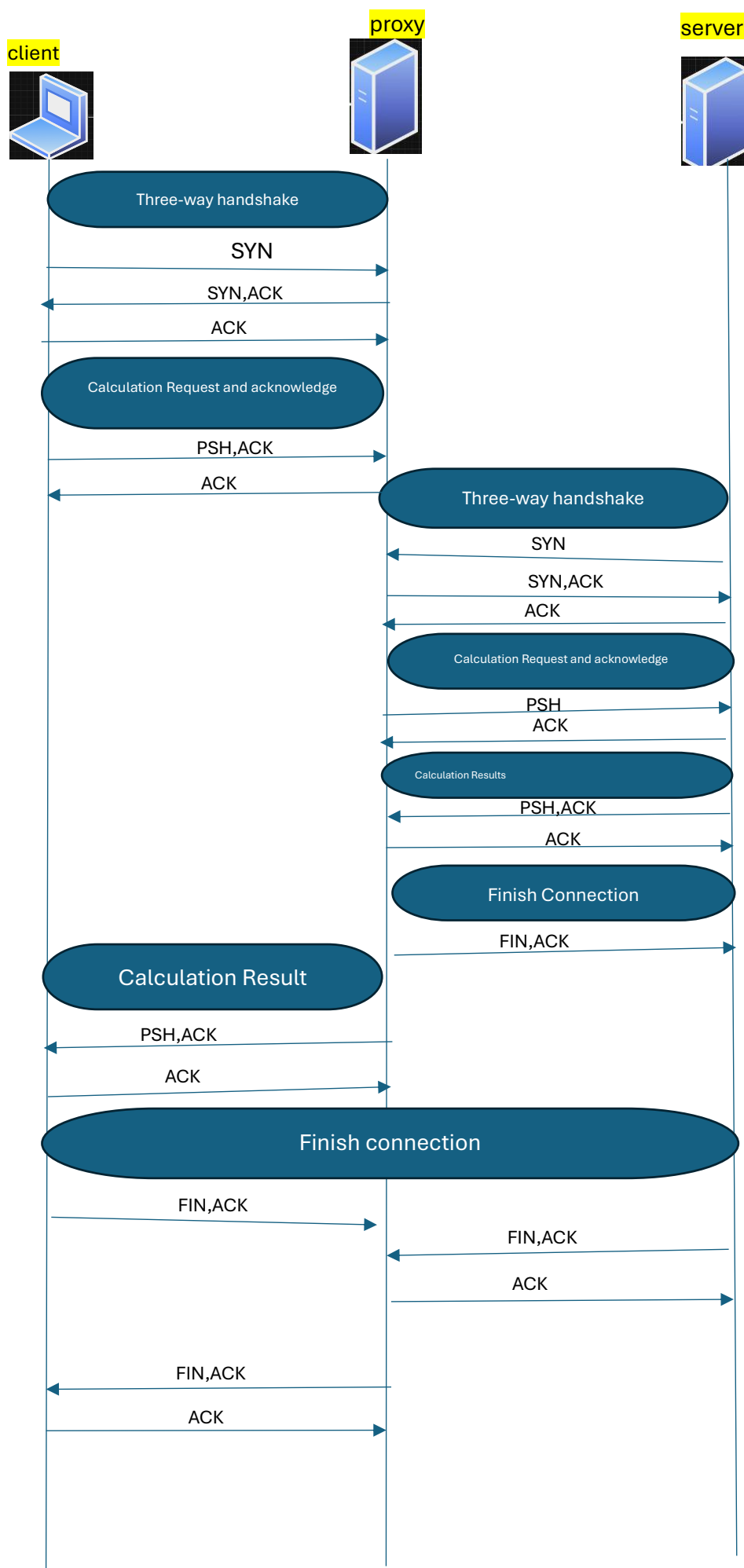
160
161     response = response.pack()
162     print(
163         f"{client_prefix} Sending response of length {len(response)} bytes")
164
165     # Send the response back to the client
166     # * Fill in start (4)
167     client_socket.sendall(response) # Sends the entire response back to the client through the client_socket.
168     # * Fill in end (4)
169

```

בשורה 167:

sendall() היא פונקצייה ששולחת את כל הנתונים שבתוך `response` ללקוח דרך הסוקט `client_socket`. זוהי השורה שבה השרת "משיב" ללקוח. בלי שורת קוד זו, הלקוח לא יקבל תשובה על הבקשה שלו.

2. **תרשימי זרימה הממחישים את מערכת המטמון עם הסברים:**
 תרשים --- < כך שהמידע אינו שמור במטמון :



תחילה הלקוח פותח קשר מסוג TCP עם הפרוקסי, בעזרת לחיצת היד המשולשת הוא שולח לו SYN דגל המראה לפרוקסי על בקשת התחברות מהלקוח, לאחר מכן הפרוקסי שולח שהוא קיבל את הבקשה שלו וגם את הכתובת שבה הוא נמצא. ולאחר מכן הלקוח מחזיר לו שהוא קיבל את התגובה שלו, כעת יש חיבור אמין בין הלקוח לפרוקסי. הלקוח שולח את בקשת החישוב שלו לפרוקסי אך הקאש של הפרוקסי ריק כלומר אין בו כלום ולכן הפרוקסי לא יכול לשרת את הלקוח ולכן הפרוקסי פותח קשר TCP עם השרת, שולח לו את בקשת החישוב והשרת מחזיר לו את התשובה לבקשה ולאחר מכן הפרוקסי מודיע לשרת שאין לו עוד בקשות לשלוח ומצידו הקשר נסגר ביניהם. כעת הפרוקסי מעביר את התשובה שהוא קיבל מהשרת ללקוח ולאחר מכן מתקיימת ההתנתקות של הלקוח מהפרוקסי ומהפרוקסי לשרת באופן סופי

הסבר כללי :

Calculation Request and Acknowledgement

מטרה: הלקוח שולח בקשה חישובית או נתונים לפרוקסי/שרת.

PSH, ACK (הלקוח שולח את הנתונים) עם PSH לדחיפה מיידית ליישום (והפרוקסי/שרת מאשר את קבלתם עם ACK) הלקוח מקבל אישור שהפרוקסי/שרת קיבל את הבקשה.

PSH מבטיח שהבקשה תועבר מיד ליישום, ACK מבטיח שהחבילה התקבלה בצורה תקינה. והACK הלקוח מקבל אישור שהפרוקסי/שרת קיבל את הבקשה.

Calculation Result

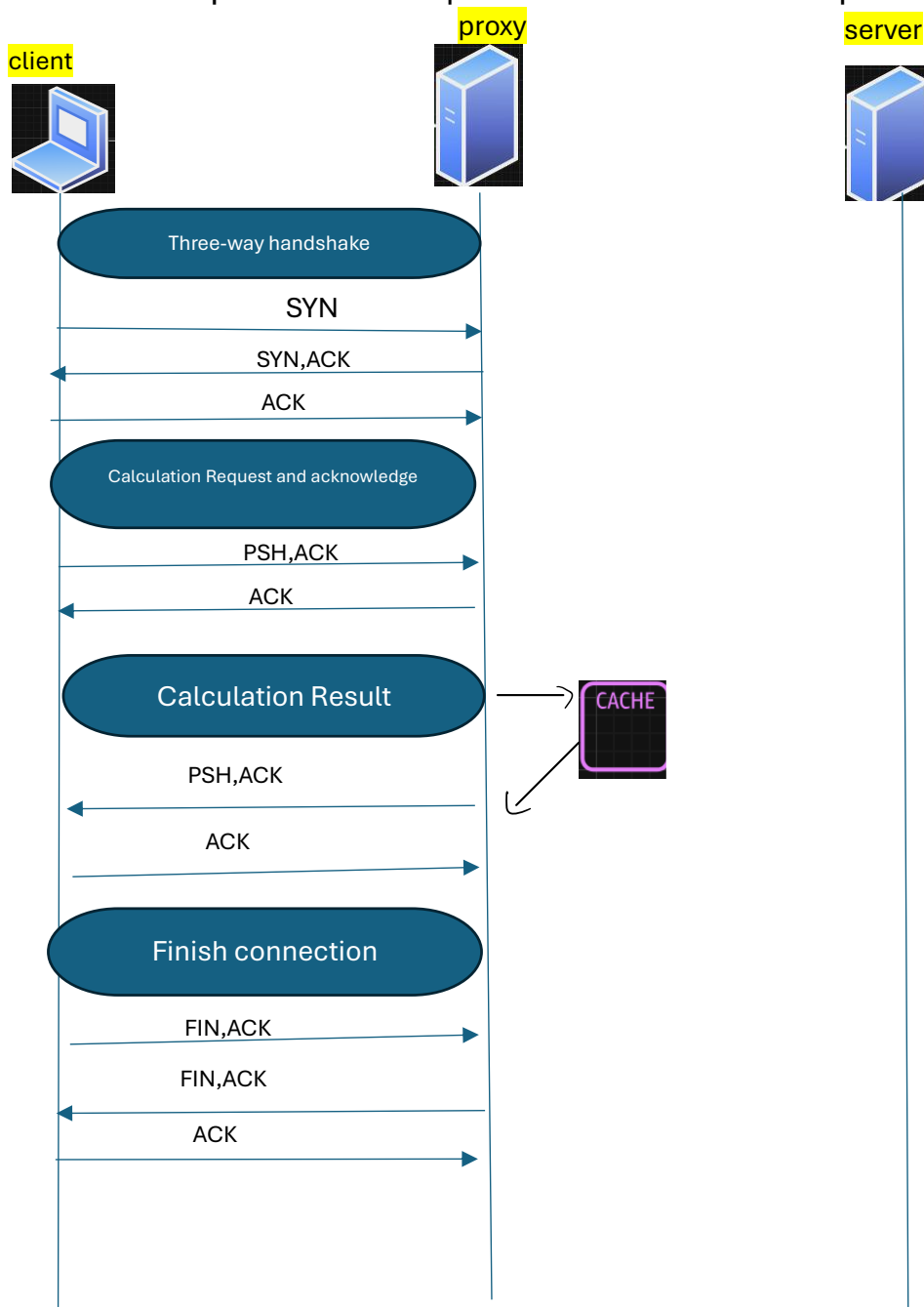
PSH, ACK השרת שולח את תוצאת החישוב לפרוקסי, שמעביר אותה ללקוח.

ACK הלקוח מאשר שקיבל את תוצאת החישוב.

הנתונים נשלחים מיד ללקוח באמצעות דגל PSH

הפרוטוקול ממשיך לבדוק שאף נתון לא אובד באמצעות אישורים. (ACK)

התרשים המייצג את התהליך כאשר יש לנו את התשובה לבקשה שמורה במטמון :



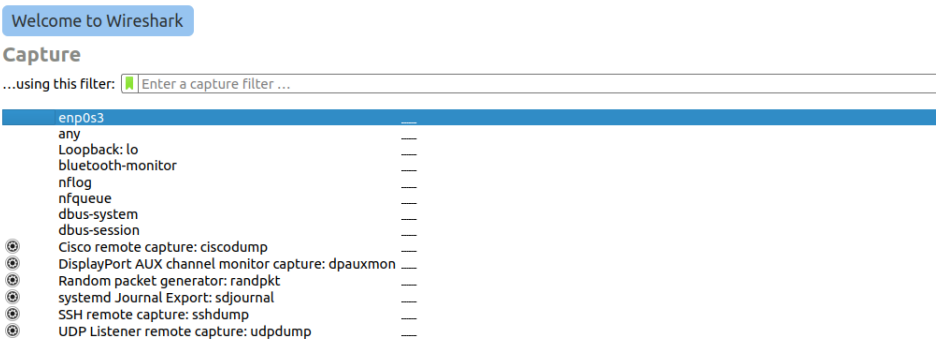
התרשים מדגים את המקרה בו הבקשה של הקליינט מוכרת על ידי הפרוקסי כלומר התשובה לבקשה של הקליינט מופיעה בקאש של הפרוקסי התהליך מאוד דומה כמו שהסברתי בהסבר למעלה,

כלומר client מנסה ליצור קשר עם הפרוקסי והוא יוצר את הקשר מסוג TCP ובעזרת לחיצת יד משולשת ובשימוש בדגלים המופעים בתרשים (ההסבר על הדגלים מופיע בהסבר על התרשים הראשון) הוא מצליח ליצור קשר אמין עם הפרוקסי וכעת הוא מוכן לשלוח בקשות. לאחר שההתחברות התבצעה בהצלחה אז הלקוח שולח לפרוקסי לבצע חישוב מסוים והפרוקסי מודיע לו שהוא קיבל את הבקשה שלו על ידי ACK. לאחר מכן הפרוקסי בודק האם יש תשובה בקאש שלו ואנחנו בתרשים שמדגים כי יש תשובה לבקשת הלקוח ולכן הפרוקסי שולף מהקאש שלו את המידע ושולח אותו בחזרה ללקוח הלקוח אישור על זה שהוא יקבל את ההודעה ולאחר מכן קורה תהליך של סגירת הקשר באמצעות הדגל FIN.

ההקלטות בwireshark:

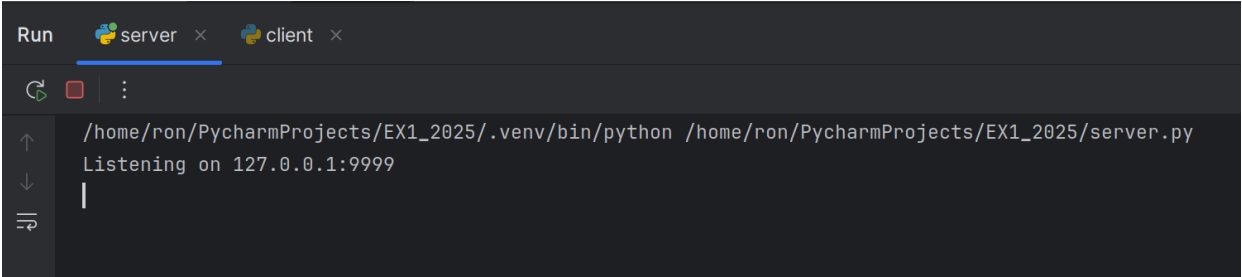
3.1 הרצת השרת והקליינט בלבד

לקוח שולח את אותה הבקשה פעמיים אחד אחרי השני ולאחר מכן שולח בקשה שונה.

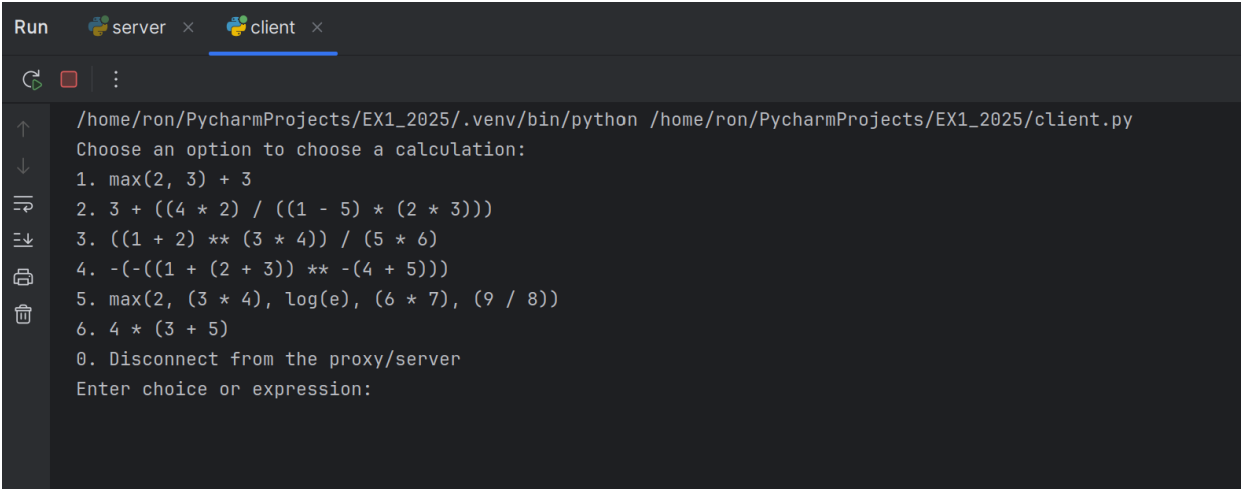


נפתח את wireshark
ונבחר ב:loopback
משום שאנחנו צריכים
להעביר הודעות
במחשב שלנו לעצמו אז
נבחר בזה .

נפעיל את השרת:



נפעיל את הקליינט:



מרשימת הביטויים יש 1-6 אפשרויות אז נניח נשלח את בקשה מספר 4 אחת אחרי השנייה

פעם ראשונה:

```
Enter choice or expression: 4
{127.0.0.1:9999} Connection established
{127.0.0.1:9999} Sending request of length 487 bytes
{127.0.0.1:9999} Got response of length 164 bytes
Result: 17714.7
Steps:
((1 + 2) ** (3 * 4)) / (5 * 6) = (3 ** (3 * 4)) / (5 * 6)
= (3 ** 12) / (5 * 6)
= 531441 / (5 * 6)
= 531441 / 30
= 17714.7
```

פעם שנייה :

```
Enter choice or expression: 4
{127.0.0.1:9999} Connection established
{127.0.0.1:9999} Sending request of length 487 bytes
{127.0.0.1:9999} Got response of length 164 bytes
Result: 17714.7
Steps:
((1 + 2) ** (3 * 4)) / (5 * 6) = (3 ** (3 * 4)) / (5 * 6)
= (3 ** 12) / (5 * 6)
= 531441 / (5 * 6)
= 531441 / 30
= 17714.7
{127.0.0.1:9999} Connection closed
```

פעם שלישית בבקשה אחרת ניוח 3 :

```
Enter choice or expression: 3
{127.0.0.1:9999} Connection established
{127.0.0.1:9999} Sending request of length 548 bytes
{127.0.0.1:9999} Got response of length 222 bytes
Result: 3.0001220703125
Steps:
3 + ((4 * 2) / ((1 - 5) ** (2 ** 3))) = 3 + (8 / ((1 - 5) ** (2 ** 3)))
= 3 + (8 / (-4 ** (2 ** 3)))
= 3 + (8 / (-4 ** 8))
= 3 + (8 / 65536)
= 3 + 0.0001220703125
= 3.0001220703125
{127.0.0.1:9999} Connection closed
```

חישוב למעשה
ה' המספר

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	47596 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989469364 TSecr=0 WS=128
2	0.000013	127.0.0.1	127.0.0.1	TCP	74	9999 → 47596 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989469364 TSecr=989469364 WS=128
3	0.000022	127.0.0.1	127.0.0.1	TCP	66	47596 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989469364 TSecr=989469364
4	0.000534	127.0.0.1	127.0.0.1	TCP	553	47596 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=989469364 TSecr=989469364 [TCP segment of a reassembled PDU]
5	0.000539	127.0.0.1	127.0.0.1	TCP	66	9999 → 47596 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=989469364 TSecr=989469364
6	0.004165	127.0.0.1	127.0.0.1	TCP	230	9999 → 47596 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=989469368 TSecr=989469364 [TCP segment of a reassembled PDU]
7	0.004206	127.0.0.1	127.0.0.1	TCP	66	47596 → 9999 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=989469368 TSecr=989469368
8	0.005177	127.0.0.1	127.0.0.1	TCP	66	47596 → 9999 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=989469369 TSecr=989469368
9	0.005222	127.0.0.1	127.0.0.1	TCP	66	9999 → 47596 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=989469370 TSecr=989469369
10	0.006536	127.0.0.1	127.0.0.1	TCP	66	47596 → 9999 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=989469370 TSecr=989469370
11	1.228971	127.0.0.1	127.0.0.1	TCP	74	47608 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989470591 TSecr=0 WS=128
12	1.228980	127.0.0.1	127.0.0.1	TCP	74	9999 → 47608 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989470591 TSecr=989470591 WS=128
13	1.228988	127.0.0.1	127.0.0.1	TCP	66	47608 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989470591 TSecr=989470591
14	1.227136	127.0.0.1	127.0.0.1	TCP	553	47608 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=989470591 TSecr=989470591 [TCP segment of a reassembled PDU]
15	1.227139	127.0.0.1	127.0.0.1	TCP	66	9999 → 47608 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=989470591 TSecr=989470591
16	1.227061	127.0.0.1	127.0.0.1	TCP	230	9999 → 47608 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=989470591 TSecr=989470591 [TCP segment of a reassembled PDU]
17	1.227712	127.0.0.1	127.0.0.1	TCP	66	47608 → 9999 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=989470591 TSecr=989470591
18	1.228240	127.0.0.1	127.0.0.1	TCP	66	47608 → 9999 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=989470592 TSecr=989470591
19	1.228855	127.0.0.1	127.0.0.1	TCP	66	9999 → 47608 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=989470592 TSecr=989470592
20	1.228867	127.0.0.1	127.0.0.1	TCP	66	47608 → 9999 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=989470592 TSecr=989470592
21	4.000033	127.0.0.1	127.0.0.1	TCP	74	47622 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989473444 TSecr=0 WS=128
22	4.000042	127.0.0.1	127.0.0.1	TCP	74	9999 → 47622 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989473444 TSecr=989473444 WS=128
23	4.000050	127.0.0.1	127.0.0.1	TCP	66	47622 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989473444 TSecr=989473444
24	4.000419	127.0.0.1	127.0.0.1	TCP	614	47622 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=548 TSval=989473444 TSecr=989473444 [TCP segment of a reassembled PDU]
25	4.000423	127.0.0.1	127.0.0.1	TCP	66	9999 → 47622 [ACK] Seq=1 Ack=549 Win=65024 Len=0 TSval=989473444 TSecr=989473444
26	4.002464	127.0.0.1	127.0.0.1	TCP	288	9999 → 47622 [PSH, ACK] Seq=1 Ack=549 Win=65024 Len=222 TSval=989473446 TSecr=989473444 [TCP segment of a reassembled PDU]
27	4.002472	127.0.0.1	127.0.0.1	TCP	66	47622 → 9999 [ACK] Seq=549 Ack=223 Win=65408 Len=0 TSval=989473446 TSecr=989473446
28	4.002559	127.0.0.1	127.0.0.1	TCP	66	47622 → 9999 [FIN, ACK] Seq=549 Ack=223 Win=65536 Len=0 TSval=989473446 TSecr=989473446
29	4.005192	127.0.0.1	127.0.0.1	TCP	66	9999 → 47622 [FIN, ACK] Seq=223 Ack=550 Win=65024 Len=0 TSval=989473449 TSecr=989473446
30	4.005206	127.0.0.1	127.0.0.1	TCP	66	47622 → 9999 [ACK] Seq=550 Ack=224 Win=65536 Len=0 TSval=989473449 TSecr=989473449

לפי הצילום מסך אפשר לראות שאין תקשורת בכלל אל ה proxy כל התקשורת בעצם מבוצעת מהלקוח לשרת ישירות .
הלקוח שלח את הבקשות הזהות פעמיים אל השרת ואת הבקשה האחרת שוב אל השרת , והפקטות שצבועות בצבע אפור מתארות פתיחה וסגירה של תקשורת בין שרת ללקוח . שמרנו את ההקלטה בשם q_3.1

3.2 הרצת השרת וגם הפרוקסי. הקלטה של הלקוח שולח את אותה הבקשה פעמיים אחד אחרי השני ולאחר מכן שולח בקשה שונה.
ראשית בקובץ client.py נשנה את ההגדרות כך שנוכל לתקשר עם הפרוקסי

```

86 if __name__ == "__main__":
87     arg_parser = argparse.ArgumentParser(description="A Calculator Client.")
88
89     arg_parser.add_argument(*name_or_flags: "-p", "--port", type=int,
90                             default=api.DEFAULT_PROXY_PORT, help="The port to connect to.")
91     arg_parser.add_argument(*name_or_flags: "-H", "--host", type=str,
92                             default=api.DEFAULT_SERVER_HOST, help="The host to connect to.")

```

נפעיל את השרת :

```

Run  server  client
:
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/server.py
Listening on 127.0.0.1:9999
|

```

נפעיל את הפרוקסי:

```

/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/proxy.py
Listening on 127.0.0.1:9998

```

נפעיל את הקליינט:

```

Run  server  proxy  client
:
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/client.py
Choose an option to choose a calculation:
1. max(2, 3) + 3
2. 3 + ((4 * 2) / ((1 - 5) * (2 * 3)))
3. ((1 + 2) ** (3 * 4)) / (5 * 6)
4. -(-(1 + (2 + 3)) ** -(4 + 5))
5. max(2, (3 * 4), log(e), (6 * 7), (9 / 8))
6. 4 * (3 + 5)
0. Disconnect from the proxy/server
Enter choice or expression: |

```

נוניח שנשלח לו פעמיים את האופציה 4

```

Enter choice or expression: 4
{127.0.0.1:9998} Connection established
{127.0.0.1:9998} Sending request of length 487 bytes
{127.0.0.1:9998} Got response of length 164 bytes
Result: 17714.7

```

ולאחר מכן נשלח את האופציה 3

```
Enter choice or expression: 3
{127.0.0.1:9998} Connection established
{127.0.0.1:9998} Sending request of length 548 bytes
{127.0.0.1:9998} Got response of length 222 bytes
Result: 3.0001220703125
Steps:
3 + ((4 * 2) / ((1 - 5) ** (2 ** 3))) = 3 + (8 / ((1 - 5) ** (2 ** 3)))
= 3 + (8 / (-4 ** (2 ** 3)))
= 3 + (8 / (-4 ** 8))
= 3 + (8 / 65536)
= 3 + 0.0001220703125
= 3.0001220703125
```

no.	Time	Source	Destination	Protocol	Length	Info
10.000000	127.0.0.1	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _ipp._tcp.local, "QM" question PTR _ippps._tcp.local, "QM" question	
21.027015	127.0.0.1	127.0.0.1	TCP	74	47978 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989747194 TSecr=0 WS=128	
31.027024	127.0.0.1	127.0.0.1	TCP	74	9998 → 47978 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989747194 TSecr=989747194 WS=128	
41.027036	127.0.0.1	127.0.0.1	TCP	66	47978 → 9998 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989747194 TSecr=989747194	
51.027039	127.0.0.1	127.0.0.1	TCP	553	47978 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=989747194 TSecr=989747194	
61.027043	127.0.0.1	127.0.0.1	TCP	66	9998 → 47978 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=989747194 TSecr=989747194	
71.028444	127.0.0.1	127.0.0.1	TCP	74	41486 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989747195 TSecr=0 WS=128	
81.028449	127.0.0.1	127.0.0.1	TCP	74	9999 → 41486 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989747195 TSecr=989747195 WS=128	
91.028454	127.0.0.1	127.0.0.1	TCP	66	41486 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989747195 TSecr=989747195	
101.028473	127.0.0.1	127.0.0.1	TCP	553	41486 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=989747195 TSecr=989747195 [TCP segment of a reassembled PDU]	
111.028475	127.0.0.1	127.0.0.1	TCP	66	9999 → 41486 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=989747195 TSecr=989747195	
121.031343	127.0.0.1	127.0.0.1	TCP	230	9999 → 41486 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=989747198 TSecr=989747195 [TCP segment of a reassembled PDU]	
131.031378	127.0.0.1	127.0.0.1	TCP	66	41486 → 9999 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=989747198 TSecr=989747198	
141.031486	127.0.0.1	127.0.0.1	TCP	66	41486 → 9999 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
151.031655	127.0.0.1	127.0.0.1	TCP	230	9998 → 47978 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=989747198 TSecr=989747194	
161.031659	127.0.0.1	127.0.0.1	TCP	66	47978 → 9998 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=989747198 TSecr=989747198	
171.031771	127.0.0.1	127.0.0.1	TCP	66	47978 → 9998 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
181.031795	127.0.0.1	127.0.0.1	TCP	66	9998 → 47978 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
191.031803	127.0.0.1	127.0.0.1	TCP	66	47978 → 9998 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
201.031950	127.0.0.1	127.0.0.1	TCP	66	9999 → 41486 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
211.031955	127.0.0.1	127.0.0.1	TCP	66	41486 → 9999 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=989747198 TSecr=989747198	
223.008078	127.0.0.1	127.0.0.1	TCP	74	48916 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989749264 TSecr=0 WS=128	
233.008089	127.0.0.1	127.0.0.1	TCP	74	9998 → 48916 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989749264 TSecr=989749264 WS=128	
243.008102	127.0.0.1	127.0.0.1	TCP	66	48916 → 9998 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989749264 TSecr=989749264	
253.008728	127.0.0.1	127.0.0.1	TCP	553	48916 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=989749265 TSecr=989749264	
263.008736	127.0.0.1	127.0.0.1	TCP	66	9998 → 48916 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=989749265 TSecr=989749265	
273.008805	127.0.0.1	127.0.0.1	TCP	230	9998 → 48916 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=989749265 TSecr=989749265	
283.008814	127.0.0.1	127.0.0.1	TCP	66	48916 → 9998 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=989749265 TSecr=989749265	
293.008880	127.0.0.1	127.0.0.1	TCP	66	48916 → 9998 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=989749265 TSecr=989749265	
303.008892	127.0.0.1	127.0.0.1	TCP	66	9998 → 48916 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=989749265 TSecr=989749265	
313.008902	127.0.0.1	127.0.0.1	TCP	66	48916 → 9998 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=989749265 TSecr=989749265	
3223.64060	127.0.0.1	224.0.0.251	MDNS	160	Standard query 0x0000 PTR _ftp._tcp.local, "QM" question PTR _nfs._tcp.local, "QM" question PTR _afpovertcp._tcp.local, "QM" question PTR _smb._tcp.local, "QM"	
3357.94533	127.0.0.1	127.0.0.1	TCP	74	59796 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989804112 TSecr=0 WS=128	
3457.94536	127.0.0.1	127.0.0.1	TCP	74	9998 → 59796 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989804112 TSecr=989804112 WS=128	

3557.94557	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	59796 → 9998 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989804112 TSecr=989804112
3657.94625	127.0.0.1	127.0.0.1	127.0.0.1	TCP	614	59796 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=548 TSval=989804113 TSecr=989804112
3757.94625	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	9998 → 59796 [ACK] Seq=1 Ack=549 Win=65024 Len=0 TSval=989804113 TSecr=989804113
3857.94631	127.0.0.1	127.0.0.1	127.0.0.1	TCP	74	60392 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=989804113 TSecr=0 WS=128
3957.94631	127.0.0.1	127.0.0.1	127.0.0.1	TCP	74	9999 → 60392 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=989804113 TSecr=989804113 WS=128
4057.94632	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	60392 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=989804113 TSecr=989804113
4157.94634	127.0.0.1	127.0.0.1	127.0.0.1	TCP	614	60392 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=548 TSval=989804113 TSecr=989804113 [TCP segment of a reassembled PDU]
4257.94634	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	9999 → 60392 [ACK] Seq=1 Ack=549 Win=65024 Len=0 TSval=989804113 TSecr=989804113
4357.94749	127.0.0.1	127.0.0.1	127.0.0.1	TCP	288	9999 → 60392 [PSH, ACK] Seq=1 Ack=549 Win=65024 Len=222 TSval=989804114 TSecr=989804113 [TCP segment of a reassembled PDU]
4457.94750	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	60392 → 9999 [ACK] Seq=549 Ack=223 Win=65408 Len=0 TSval=989804114 TSecr=989804114
4557.94755	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	60392 → 9999 [FIN, ACK] Seq=549 Ack=223 Win=65536 Len=0 TSval=989804114 TSecr=989804114
4657.94758	127.0.0.1	127.0.0.1	127.0.0.1	TCP	288	9998 → 59796 [PSH, ACK] Seq=1 Ack=549 Win=65024 Len=222 TSval=989804114 TSecr=989804113
4757.94759	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	59796 → 9998 [ACK] Seq=549 Ack=223 Win=65408 Len=0 TSval=989804114 TSecr=989804114
4857.94767	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	59796 → 9998 [FIN, ACK] Seq=549 Ack=223 Win=65536 Len=0 TSval=989804114 TSecr=989804114
4957.94769	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	9998 → 59796 [FIN, ACK] Seq=223 Ack=550 Win=65024 Len=0 TSval=989804114 TSecr=989804114
5057.94770	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	59796 → 9998 [ACK] Seq=550 Ack=224 Win=65536 Len=0 TSval=989804114 TSecr=989804114
5157.94775	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	9999 → 60392 [FIN, ACK] Seq=223 Ack=550 Win=65024 Len=0 TSval=989804114 TSecr=989804114
5257.94775	127.0.0.1	127.0.0.1	127.0.0.1	TCP	66	60392 → 9999 [ACK] Seq=550 Ack=224 Win=65536 Len=0 TSval=989804114 TSecr=989804114

ששלחנו את הבקשה הראשונה המטמון היה ריק ולכן הפרוקסי פנה לשרת כדי לקבל תשובה .
השרת החזיר את התשובה לפרוקסי והפרוקסי החזיר תשובה לקליינט. בקשת החישוב השנייה
תשלח כך שהמטמון לא ריק ולכן הפרוקסי יחזיר את התוצאה ללקוח מבלי לפנות לשרת .
(אפשר להבחין בין כל המקרים בעזרת מספר הסרסר)

שמרנו את ההקלטה בשם q_3.2

3.3

הרצה של השרת ואת הפרוקסי. הקלטה של הלקוח שולח בקשה, לאחר מכן סיגרו את השרת,
תשלחו את אותה הבקשה עוד הפעם ולאחר מכן שילחו בקשה שונה.
נפעיל את השרת:

```
Run server x proxy x client x
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/server.py
Listening on 127.0.0.1:9999
```

נפעיל את הפרוקסי :

```
Run server x proxy x client x
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/proxy.py
Listening on 127.0.0.1:9998
```

וגם את הקליינט:

```
Run server x proxy x client x
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/client.py
Choose an option to choose a calculation:
1. max(2, 3) + 3
2. 3 + ((4 * 2) / ((1 - 5) * (2 * 3)))
3. ((1 + 2) ** (3 * 4)) / (5 * 6)
4. -(-(1 + (2 + 3)) ** -(4 + 5)))
5. max(2, (3 * 4), log(e), (6 * 7), (9 / 8))
6. 4 * (3 + 5)
0. Disconnect from the proxy/server
Enter choice or expression: 4
```

ונניח שאנחנו רוצים לשלוח את 4 לשרת .

No.	Time	Source	Destination	Protocol	Length	Info
40	0.000311...	127.0.0.1	127.0.0.1	TCP	553	35972 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=540106641 TSecr=540106641
50	0.000315...	127.0.0.1	127.0.0.1	TCP	66	9998 → 35972 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=540106641 TSecr=540106641
60	0.002144...	127.0.0.1	127.0.0.1	TCP	74	35146 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=540106643 TSecr=0 WS=128
70	0.002149...	127.0.0.1	127.0.0.1	TCP	74	9999 → 35146 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=540106643 TSecr=540106643 WS=128
80	0.002153...	127.0.0.1	127.0.0.1	TCP	66	35146 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=540106643 TSecr=540106643
90	0.002281...	127.0.0.1	127.0.0.1	TCP	553	35146 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=540106643 TSecr=540106643 [TCP segment of a reassembled PDU]
100	0.002283...	127.0.0.1	127.0.0.1	TCP	66	9999 → 35146 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=540106643 TSecr=540106643
110	0.003107...	127.0.0.1	127.0.0.1	TCP	230	9999 → 35146 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=540106644 TSecr=540106643 [TCP segment of a reassembled PDU]
120	0.003112...	127.0.0.1	127.0.0.1	TCP	66	35146 → 9999 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=540106644 TSecr=540106644
130	0.003165...	127.0.0.1	127.0.0.1	TCP	66	35146 → 9999 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=540106644 TSecr=540106644
140	0.003250...	127.0.0.1	127.0.0.1	TCP	230	9999 → 35972 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=540106644 TSecr=540106641
150	0.003254...	127.0.0.1	127.0.0.1	TCP	66	35972 → 9998 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=540106644 TSecr=540106644
160	0.003407...	127.0.0.1	127.0.0.1	TCP	66	9999 → 35146 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=540106645 TSecr=540106644
170	0.003415...	127.0.0.1	127.0.0.1	TCP	66	35146 → 9999 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=540106645 TSecr=540106645
180	0.003541...	127.0.0.1	127.0.0.1	TCP	66	35972 → 9998 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=540106645 TSecr=540106644
190	0.003569...	127.0.0.1	127.0.0.1	TCP	66	9998 → 35972 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=540106645 TSecr=540106645
200	0.003573...	127.0.0.1	127.0.0.1	TCP	66	35972 → 9998 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=540106645 TSecr=540106645

נסגור את השרת

```
Run server x proxy x client x
/home/ron/PycharmProjects/EX1_2025/.venv/bin/python /home/ron/PycharmProjects/EX1_2025/server.py
Listening on 127.0.0.1:9999
Connection established with 127.0.0.1:35146
{127.0.0.1:35146} Got request of length 487 bytes
{127.0.0.1:35146} Sending response of length 164 bytes
{127.0.0.1:35146} Connection closed
Shutting down...
Process finished with exit code 0
```

נפעיל את הבקשה מהקליינט שוב :

29 72.60238...	127.0.0.1	127.0.0.1	TCP	74 38464 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=540179243 TSecr=0 WS=128
30 72.60239...	127.0.0.1	127.0.0.1	TCP	74 9998 → 38464 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=540179243 TSecr=540179243 WS=128
31 72.60239...	127.0.0.1	127.0.0.1	TCP	66 38464 → 9998 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=540179244 TSecr=540179243
32 72.60467...	127.0.0.1	127.0.0.1	TCP	553 38464 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=487 TSval=540179246 TSecr=540179243
33 72.60467...	127.0.0.1	127.0.0.1	TCP	66 9998 → 38464 [ACK] Seq=1 Ack=488 Win=65024 Len=0 TSval=540179246 TSecr=540179246
34 72.60474...	127.0.0.1	127.0.0.1	TCP	230 9998 → 38464 [PSH, ACK] Seq=1 Ack=488 Win=65536 Len=164 TSval=540179246 TSecr=540179246
35 72.60478...	127.0.0.1	127.0.0.1	TCP	66 38464 → 9998 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=540179246 TSecr=540179246
36 72.60515...	127.0.0.1	127.0.0.1	TCP	66 38464 → 9998 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=540179246 TSecr=540179246
37 72.60528...	127.0.0.1	127.0.0.1	TCP	66 9998 → 38464 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=540179246 TSecr=540179246
38 72.60529...	127.0.0.1	127.0.0.1	TCP	66 38464 → 9998 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=540179246 TSecr=540179246
39 92.07419...	127.0.0.1	224.0.0.251	MDNS	87 Standard query 0x0000 PTR _ipp._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question

לאחר מכן ביקשנו בקשה ששונה מ4 למשל 3

```

Run server proxy client
= 531441 / (5 * 6)
= 531441 / 30
= 17714.7
{127.0.0.1:9998} Connection closed
Enter choice or expression: 3
{127.0.0.1:9998} Connection established
{127.0.0.1:9998} Sending request of length 548 bytes
{127.0.0.1:9998} Got response of length 170 bytes
{127.0.0.1:9998} Got error: ('Internal proxy error', CalculatorServerError('Connection refused by server and the request was not in the cache/it was stale'))

```

ונשים לב כי גם בטרמינל וגם בwireSharke הלקוח שלנו לא מקבל מידע עקב חוסר יכולת לגשת לשרת:

```

Run server proxy client
# that the client is willing to accept (in seconds)
{127.0.0.1:35972} Connection closed
{127.0.0.1:38464} Connected established
{127.0.0.1:38464} Got request of length 487 bytes
{127.0.0.1:38464} Cache hit ,server time remaining: inf, client time remaining: inf
{127.0.0.1:38464} Sending response of length 164 bytes
{127.0.0.1:38464} Connection closed
{127.0.0.1:37792} Connected established
{127.0.0.1:37792} Got request of length 548 bytes
Unexpected server error: Connection refused by server and the request was not in the cache/it was stale
{127.0.0.1:37792} Connection closed

```

No.	Time	Source	Destination	Protocol	Length	Info
35	72.60476...	127.0.0.1	127.0.0.1	TCP	66	38464 → 9998 [ACK] Seq=488 Ack=165 Win=65408 Len=0 TSval=540179246 TSecr=540179246
36	72.60515...	127.0.0.1	127.0.0.1	TCP	66	38464 → 9998 [FIN, ACK] Seq=488 Ack=165 Win=65536 Len=0 TSval=540179246 TSecr=540179246
37	72.60528...	127.0.0.1	127.0.0.1	TCP	66	9998 → 38464 [FIN, ACK] Seq=165 Ack=489 Win=65536 Len=0 TSval=540179246 TSecr=540179246
38	72.60529...	127.0.0.1	127.0.0.1	TCP	66	38464 → 9998 [ACK] Seq=489 Ack=166 Win=65536 Len=0 TSval=540179246 TSecr=540179246
39	92.07419...	127.0.0.1	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _ipp._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
40	134.8150...	127.0.0.1	127.0.0.1	TCP	74	37792 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=540241458 TSecr=0 WS=128
41	134.8150...	127.0.0.1	127.0.0.1	TCP	74	9998 → 37792 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=540241458 TSecr=540241458 WS=128
42	134.8152...	127.0.0.1	127.0.0.1	TCP	66	37792 → 9998 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=540241458 TSecr=540241458
43	134.8152...	127.0.0.1	127.0.0.1	TCP	614	37792 → 9998 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=548 TSval=540241458 TSecr=540241458
44	134.8152...	127.0.0.1	127.0.0.1	TCP	66	9998 → 37792 [ACK] Seq=1 Ack=549 Win=65024 Len=0 TSval=540241458 TSecr=540241458
45	134.8176...	127.0.0.1	127.0.0.1	TCP	74	40258 → 9998 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=540241458 TSecr=0 WS=128
46	134.8176...	127.0.0.1	127.0.0.1	TCP	549	9998 → 40258 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=540241458 TSecr=540241458 WS=128
47	134.8171...	127.0.0.1	127.0.0.1	TCP	230	9998 → 37792 [PSH, ACK] Seq=1 Ack=549 Win=65024 Len=170 TSval=540241458 TSecr=540241458
48	134.8172...	127.0.0.1	127.0.0.1	TCP	66	37792 → 9998 [ACK] Seq=549 Ack=171 Win=65408 Len=0 TSval=540241458 TSecr=540241458
49	134.8176...	127.0.0.1	127.0.0.1	TCP	66	37792 → 9998 [FIN, ACK] Seq=549 Ack=171 Win=65536 Len=0 TSval=540241459 TSecr=540241458
50	134.8186...	127.0.0.1	127.0.0.1	TCP	66	9998 → 37792 [FIN, ACK] Seq=171 Ack=550 Win=65024 Len=0 TSval=540241459 TSecr=540241459
51	134.8180...	127.0.0.1	127.0.0.1	TCP	66	37792 → 9998 [ACK] Seq=550 Ack=172 Win=65536 Len=0 TSval=540241459 TSecr=540241459

שמרנו את ההקלטה בשם q_3.3

נשלחה בקשה לפרוקסי המטמון היה ריק ולכן נשלחה בקשה לשרת. השרת העביר את התשובה לפרוקסי והפרוקסי העביר את התשובה ללקוח. אחרי שלב זה סגרנו את השרת. שלחנו עוד פעם את אותה הבקשה לפרוקסי. הפעם התשובה נמצאה במטמון והפרוקסי שלח את התשובה בחזרה ללקוח. בפעם השלישית שהלקוח ביקש בקשה שונה מהשניים הראשונות. הבקשה נשלחה לפרוקסי. במטמון לא היה פתרון של השאלה ולכן הוא ניסה לפנות לשרת. אבל השרת כבוי ולכן נתקבלה פקטה מסוג RST אשר מסמנת לפרוקסי שיש בעיה בשרת והחיבור אימו נסגר (מופיע באדום). לאחר מכן הפרוקסי שולח הודעת טקסט ללקוח שהייתה בעיה בטיפול בבקשה שלו, כפי שניתן לראות בתצלומי המסך מהטרמינל ומהwireshark.

שאלות פתוחות :



נתון: F : אורך הליקס
 מספר הנתונים: X : מספר הנתונים
 אורך הליקס של כל נתון: d
 קצב העברת הנתונים: R
 אורך קווי הליקס: x
 מרחק הנתונים: S

חישוב RTT: נחשב כמה זמן ייקח "לפזר" הנתונים מהמקור אל הסינק.
 1. זמן $\frac{x}{S}$ כי הוא צריך להעביר את הנתונים.
 2. כאשר הנתונים מגיעים אל A_1 יורה פיו"ל של d נוסף אל הנתונים והוא ממשיך להעבירם אל A_2 .
 3. $\frac{x}{S}$ נוסף. לכן הזמן שצריך לכל הנתונים להגיע אל הסינק הוא:
 $\frac{x}{S} + d + \frac{x}{S} + d + \frac{x}{S} = 3\frac{x}{S} + 2d$

אילו RTT נחשב את הזמן שצריך להעביר את הנתונים מהמקור אל הסינק.
 $RTT = 2 \cdot (3\frac{x}{S} + 2d) = 6\frac{x}{S} + 4d$

1. HTTP Non persistent, כלומר כל פעם שצריך להעביר נתונים צריך לבצע RTT נוסף.
 לכן: $RTT = 2 \cdot (3\frac{x}{S} + 2d) = 6\frac{x}{S} + 4d$

$$T_c = 2RTT + \frac{F}{R} + X[2RTT + \frac{4F}{R}] =$$

$$2 \left(\frac{6x}{S} + 4d \right) + \frac{F}{R} + X \left[2 \left(\frac{6x}{S} + 4d \right) + \frac{4F}{R} \right]$$

$$= \frac{12x}{S} + 8d + \frac{F}{R} + \frac{12xX}{S} + 4dX + \frac{4FX}{R}$$

$$= \frac{12x}{S} (1+X) + 4d(2+X) + \frac{F}{R} (1+4X)$$

כל הנתונים של P צריכים להגיע אל הסינק.
 כל הנתונים של P צריכים להגיע אל הסינק.
 כל הנתונים של P צריכים להגיע אל הסינק.

מהירות גלי תצפית נמדדת בעזרת, HTTP Persistent .2

$$\begin{aligned}
 T_0 &= 2RTT + \frac{F}{R} + X \left[RTT + \frac{4F}{R} \right] \\
 &= 2 \cdot (6\frac{x}{5} + 4d) + \frac{F}{R} + X \left[6\frac{x}{5} + 2d + \frac{4F}{R} \right] \\
 &= \frac{12x}{5} + 8d + \frac{F}{R} + \frac{6x}{5}X + 2dX + \frac{4F}{R}X \\
 &= \frac{6}{5}x(2+X) + 2d(4+X) + \frac{F}{R}(1+4X)
 \end{aligned}$$

מדובר ב: שני פיקטורים
 הראשון הוא המרחק בין השרת לבין הלקוח
 השני הוא המרחק בין השרת לבין השרת
 המרחק בין השרת לבין השרת

מהירות גלי תצפית נמדדת בעזרת, HTTP Persistent Pipelining

זאת תצפית נמדדת בעזרת, HTTP Persistent Pipelining

$$\begin{aligned}
 T_1 &= 2RTT + \frac{F}{R} + RTT + X \left[\frac{4F}{R} \right] \\
 &= 2(6\frac{x}{5} + 4d) + \frac{F}{R} + (6\frac{x}{5} + 4d) + \frac{4F}{R}X \\
 &= \frac{12x}{5} + 8d + \frac{F}{R} + \frac{6x}{5} + 4d + \frac{4F}{R}X = \\
 &= \frac{18}{5}x + 12d + \frac{F}{R}(1+4X)
 \end{aligned}$$

מדובר ב: שני פיקטורים
 הראשון הוא המרחק בין השרת לבין הלקוח
 השני הוא המרחק בין השרת לבין השרת
 המרחק בין השרת לבין השרת

1ב. ההבדלים בין כל אחת מהשיטות שמוזכרות בסעיף א' הם:
 HTTP non-persistent – אנחנו שולחים בקשת התחברות מסוג tcp, שולחים אובייקט אחד ולאחר מכן מתנתקים מה tcp. כלומר הקשר נסגר לאחר טיפול בבקשה ולכל היותר אובייקט אחד נשלח על ידי החיבור שיצרנו.

Persistent HTTP – פתיחת התחברות TCP, שולח כמה אובייקטים ביחד ולאחר מכן מתנתקים מחיבור ה TCP כלומר הקשר נשאר פתוח לאחר טיפול בבקשה ואובייקטים רבים יכולים להישלח על ידי חיבור אחד.

HTTP persistent with pipelining – בחיבור מסוג זה, אנחנו שולחים כמה בקשות ביחד על ידי חיבור TCP אחד ואז נקבל את התשובות עבורן ביחד

1ג. HTTP POST – דורשת ששרת האינטרנט יקבל את הנתונים הכלולים בגוף הודעת הבקשה, על מנת שהשרת יוכל לאחסן אותם.

HTTP GET – שיטה זו משמשת לבקשת מידע משרת. הלקוח (דפדפן או אפליקציה) שולח בקשה לשרת לקבל נתונים כלומר משמש לבקשת נתונים ממקור מוגדר, בקשות GET לרוב נשמרות במטמון של הדפדפן, ומאפשרות טעינה מהירה יותר במקרים של בקשות חוזרות.

ב.המסמך שיהיה ארוך ומפורט על 1 GByte

לפי אורך המסמך צריך להכין 30 GByte קובץ

$$307.2 \text{ sec} = \frac{30,720}{100} = \frac{30 \cdot 1024}{100} = \frac{30 \text{ GByte}}{100 \text{ MByte}} = \frac{\text{סגור תפיל}}{\text{קצת בלילה ופיל}} \leftarrow$$

נבדוק את קצת הודעת ההגדרה P. הנושא קובץ המסמך שיהיה ארוך ומפורט על 1 GByte
מסמך קצת מורחב ופיל הוא מיליון וזה קובץ מה שקבע

$$\frac{1 \text{ GByte}}{4 \text{ MByte/sec}} = \frac{1024}{4} = 256 \text{ sec}$$

אנחנו קובעים את הזמן שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte
הזמן המיועד יהיה שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte
הזמן המיועד יהיה שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte

$$D_{\text{P2P}} = \max(256, 307.2) = 307.2 \text{ sec}$$

קצת P2P ← מסמך שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte
הזמן המיועד יהיה שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte
הזמן המיועד יהיה שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte

$$\frac{1 \text{ GByte}}{10 \text{ MByte/sec}} = \frac{1024}{10} = 102.4 \text{ sec}$$

הזמן המיועד יהיה שיהיה ארוך ומפורט על 1 GByte. המסמך שיהיה ארוך ומפורט על 1 GByte

נבדוק את 30 המסמך הארוך, אמרנו צריך להכין 30 GByte קובץ

$$\frac{30 \text{ GByte}}{130 \text{ MByte}} = \frac{30 \cdot 1024}{130} = 236.307 \text{ sec} \quad 10 + 5 \cdot 10 + 2 \cdot 10 = 130$$

$$D_{\text{P2P}} = \max(102.4, 256, 236.307) = 256 \text{ sec}$$

אמרו חז"ל שבעה חלקים הן קצת ירוח סבב שער כי אנוניו לתקו"ל זה המקור שנתן
לומר שער שער ירוח סבב שער ולכן נתייחס יתן לומר שער הן **קצת**
ס. כי שער השער נשקף כי לאחר המעשים בסוף ה' הארץ השבט הן קצת השער
המקור ל' השער, ואם הוא ירוח יתן שער שער הן הן
הקצת. כוונתו מה שנתן משהם שער שער השער השער ל' השער
שער קצת השער ל' השער שער ירוח, והשער שער קצת שער
מה שנתן שער השער השער ל' השער שער שער (שער שער) שער
השער שער שער שער.

[illegible]

2. הקס'ל המורכב משני קטבים P_2P :

1. כדאי להשתמש ב-VPN כדי להגן על פרטיותך ונתונים שלך.
 (לדוגמה: NordVPN, ExpressVPN, ProtonVPN)
 ניתן להשתמש ב-BitTorrent כדי להוריד קבצים ישירות מהרשת.
 יש להיזהר היטב מהפרטים שניתנים באתר.

מה עולו בענין זה ומה עולו
 מה עולו מה עולו מה עולו

~~Das~~ Problem deines Pils wurde identifiziert. Das Problem ist BitTorrent. Es wird
nicht nur das, sondern Pils als Pils, das Problem mit Pils, dass
das Problem ist, dass es ist.

[illegible]

אשר לא היה, מן הסתם היה זה המצב לפני.
לפיכך יש להבחין בין שני סוגי בעיות.
הראשונה היא בעיה של זיהוי מילים.

השנייה: זיהוי ה- chatGPT של שני סוגי בעיות, כי הם אינם
ההבחנה בין שני סוגי בעיות.

השנייה